Project Report

CAS 782 (2026W)

Vehicle Distributed Online Parameter Estimation and Auto-Park Digital Twin

Author
Samuel Khzym khzyms@mcmaster.ca khzyms

April 6, 2026

Original template credits: Samuel Zurowski and Charles Barone. Adapted for McMaster CAS by Dr. Istvan David.

1 Executive summary

This project aims to develop a digital twin for the automotive space which runs in the cloud and which the physical twin (one-tenth scale vehicle) communicates with wirelessly. The digital twin periodically receives vehicle kinematics data continuously from the physical twin during manual driving to update a vehicle dynamics model. This involves vehicle speed, acceleration, yaw rate, and GPS location. This allows for continuous updates to the vehicle model parameters to allow for any deviations in vehicle parameters (mass, tire pressure, etc). It will also evaluate the tolerance of the digital twin to perform well under an unreliable transmission medium (wireless).

Once the physical twin is ready to do an automated parking maneuver, which is generally very compute and memory heavy, it will transmit its information to the digital twin. This involves image and point cloud data from the onboard camera and LiDAR. The digital twin will then determine an optimal trajectory for the physical twin to follow and transmit said trajectory to the physical twin, which will then attempt to follow the provided trajectory.

The intention of the majority of the work is to go towards my Master's thesis, and I would be interested in writing a journal paper on the digital twin system design and fault-tolerant properties of the system.

All code and analysis the author has written for the purposes of this project can be found at the following GitHub link: `https://github.com/SamKhzym/cloud-auto-park`.

2 Scope and requirements

2.1 Scope

The scope of the project will be broken up into a physical twin (the vehicle), and the digital twin (cloud-based application). The physical twin shall be implemented in two domains, simulation and in-vehicle.

The first domain for the physical twin is in simulation, where a virtual vehicle model provided by creators of the F1TENTH framework with variable dynamics parameters. The virtual vehicle model shall represent the physical twin and periodically communicate its state to the digital twin while manually driving. The state of the simulated physical twin will consist of its GPS location, speed, yaw rate, acceleration, and steering wheel angle. The digital twin will periodically re-optimize the vehicle parameters on the most recently received vehicle data. Once the auto-park maneuver starts, the simulated vehicle shall send the camera and LiDAR data to the digital twin, which will find the obstacles using SLAM and determine an optimal path for the simulated vehicle to follow with the best vehicle dynamics parameters determined by the parameter optimizer. The simulation will also contain a setting to drop a percentage of incoming packets to the digital twin, to simulate the unreliability of the medium in practical applications.

The second domain for the physical twin is in-vehicle, which will use McSCert's Mini M-City one-tenth scale vehicles. The vehicle shall report the same information as the simulated phys-

ical twin, with the exception of using a localization technology such as AprilTags to determine the global position of the vehicle in the room in the absence of GPS information.

The digital twin shall run on a server which will receive the periodic information from the digital twin, and periodically solve an optimization problem to update the vehicle dynamics parameters. It shall use a timeseries database to store and retrieve previous dynamics information as well. During the parking maneuver, the digital twin shall receive the LiDAR and camera data from the physical twin (either real or simulated), run a path planning algorithm to determine the trajectory to follow, then transmit the path to follow and the vehicle parameters to the physical twin for it to execute the commands to follow the trajectory with its own controller.

2.2 Requirements

Physical Twin Requirements:

- PHYS-1: The physical twin shall be controllable by a human operator using a game controller.
  - PHYS-1.1: The longitudinal motion shall be controllable by the left joystick along the vertical axis.
  - PHYS-1.2: The steering angle shall be controllable by the right joystick along the horizontal axis.
  - PHYS-1.3: To transition to autonomous mode for auto-parking, the "MODE" button on the controller shall be toggled on.
- PHYS-2: The physical twin shall have an auto-park feature.
  - PHYS-2.1: The physical twin shall detect a desired parking pose in the environment by finding a red piece of paper on the ground which is the same size as the vehicle.
  - PHYS-2.2: The physical twin shall transmit LiDAR and camera data to the digital twin to perform obstacle localization.
  - PHYS-2.3: The physical twin shall be able to follow a trajectory provided by the digital twin to within 3cm accuracy.
  - PHYS-2.4: The feature can be overridden by the operator at any point by pressing the "MODE" button on the controller.
- PHYS-4: The physical twin shall transmit IMU/GPS sensor data to the digital twin during manual driving at 20Hz.

Digital Twin Requirements:

- DIGI-1: The digital twin shall periodically save sensor data from the physical twin to a database for further processing at 20Hz.

- DIGI-2: The digital twin shall periodically update its estimate of the vehicle dynamics parameters based on incoming sensor data.
  - PHYS-2.1: The digital twin shall estimate the vehicle's mass, cornering stiffnesses, and yaw inertia to within 10% accuracy.
  - PHYS-2.2: The digital twin shall not perform the parameter update if there is insufficient or inconsistent data from the physical twin, considered anything more than 20% data loss.
- DIGI-3: The digital twin shall be able to localize the poses of obstacles in the environment from the LiDAR and camera data provided by the physical twin.
- DIGI-4: The digital twin shall be able to localize the poses of obstacles in the environment from the LiDAR and camera data provided by the physical twin to within 5cm accuracy.
- DIGI-5: The digital twin shall be able to determine the location of the desired pose by detecting the red paper in the camera frame to within 5cm accuracy.
- DIGI-6: The digital twin shall be able to calculate a trajectory to move the vehicle from its initial pose to the desired final pose.

3 Design and schedule

3.1 Architecture

Figure 1 shows the high-level architecture of the vehicle (physical twin) and the digital twin.

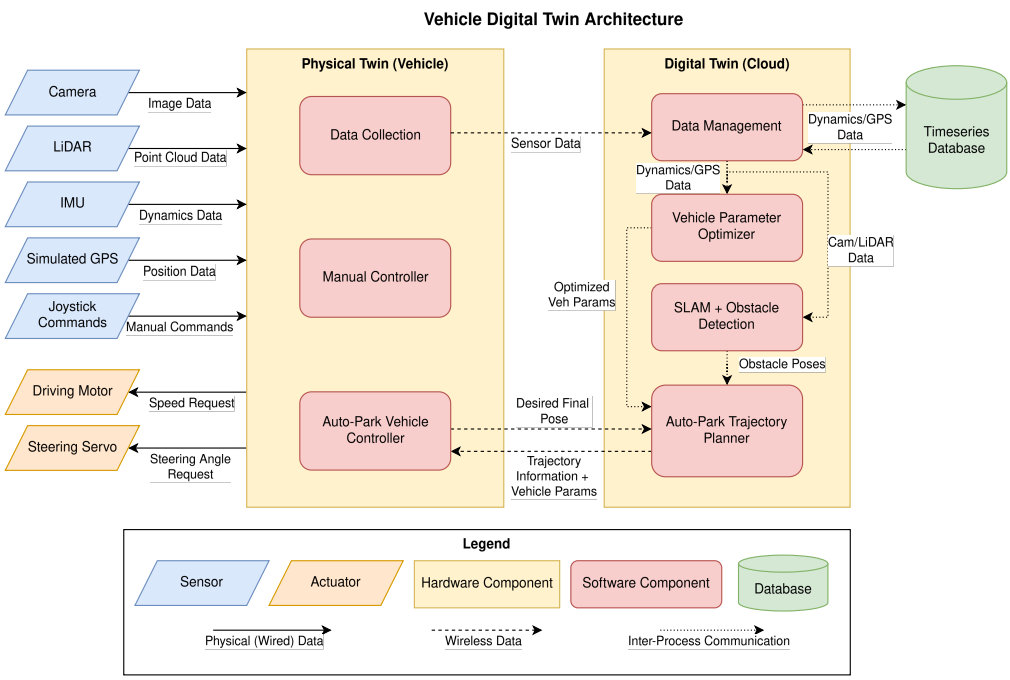


Figure 1: Vehicle Digital Twin Architecture

3.1.1 Hardware

There are two primary hardware platforms which will be used in this project.

- The physical twin vehicle will be running on the F1TENTH one-tenth scale autonomous vehicle stack. It is equipped with a camera, 2D LiDAR, and IMU sensors. The primary compute unit where the vehicle-level software will be deployed is an NVIDIA Jetson Nano.
- The digital twin will be running on a server which can be accessed wirelessly. For the purposes of this project, it will be run locally on my machine, but this could be deployed on a cloud provider as well.

3.1.2 Software

The project is broken up into seven unique software components, three of which run on the physical twin, and four of which run on the digital twin. Each of these software components shall be implemented as a ROS2 node in Python or C++.

The software components on the physical twin are as follows:

- The Data Collection component will take all sensor data from the connected sensors on the vehicle as inputs and is responsible for transmitting it to the Data Management component on the digital twin.
- The Manual Controller will be listening to the connected joystick and actuate the drive/steering motor based on the operator commands.

- The Auto-Park Vehicle Controller will transmit the desired pose to the digital twin and receive the trajectory to follow and updated vehicle dynamics parameters. It will then actuate the driving and steering motors to follow the provided trajectory.

The software components on the digital twin are as follows:

- The Data Management component will receive the sensor data from the physical twin and save them to a timeseries database which can be queried by timestamps to get all data from specific time regions.
- The Vehicle Parameter Optimizer will take the IMU information and the GPS information from the physical twin and periodically run an optimization sweep to determine the most accurate vehicle dynamics parameters. This will be running the dynamic vehicle bicycle model, which requires the vehicle mass, yaw intertia, cornering stiffnesses, and distances from center of gravity to axles.
- The SLAM + Obstacle Detection component will take the LiDAR and camera data from the physical twin and run a SLAM algorithm to localize all the obstacles in the vehicle's environment.
- The Auto-Park Trajectory Planner will take in the obstacle poses from the obstacle detection component and the desired final pose from the physical twin, then send them to the trajectory planner on the digital twin. When it receives the trajectory to follow and the most up-to-date vehicle dynamics parameters, it will generate the trajectory for the physical twin to follow.

3.1.3 Middleware

All inter-process communication from within each hardware component shall be handled as ROS2 topics. The sensor data during the manual driving phase shall be transmitted as a UDP stream since it is not required for all packets to make it to the server. During an auto-park maneuver, the transmission protocol shall switch to TCP since packet loss is more detrimental in that environment.

3.2 Technology

Each software component will be implemented as a ROS2 node (either C++ or Python), with the inter-process signals using ROS2 topics. If more efficient communication is required for the image or LiDAR data to be transmitted wirelessly, a framework such as ImageZMQ may be considered as well. The timeseries database shall use a timeseries-data oriented extension to PostgreSQL called TimescaleDB.

3.3 Deployment plans

The artifacts for the physcial twin shall be deployed onto the onboard compute unit on the F1TENTH vehicle, the NVIDIA Jetson Orin Nano, and a ROS2 launcher shall manage running all the components together. The components of the digital twin shall run on my laptop and shall also be deployed using the ROS2 launcher.

3.4 Schedule

- Week of Feb 15: Begin implementation of Data Collection, Manual Controller, and Auto-Park Vehicle Controller. Begin setup of simulated physical twin and interfaces between physical twin and digital twin.
- Week of Feb 22: Begin setup of TimescaleDB database, Data Management component, and vehicle parameter optimizer. Perform experiements with simulated vehicle to see if optimizer is able to effectively detect changes in vehicle parameters.
- Week of Mar 1: Begin development of SLAM and obstacle detection algorithm with simulated vehicle. Test with virtualized obstacles.
- Week of Mar 8: Begin development of Auto-Park Trajectory Planner, test end to end auto-park stack in simulation.
- Week of Mar 15: Transition to deploying the software onto the real physical twin in the McSCert lab. Begin final report writeup.
- Mar 22: Submit final report.

3.4.1 Addendum for Final Submission

This final submission has de-scoped some of the work outlined in the project plans, although the author intends to see the project through to the end after the class is completed. Notably, this includes dropping the deployment on the real physical twin and doing all performance analysis on the simulated digital twin temporarily. Furthermore, the SLAM + Obstacle Detection software component has been de-scoped and currently just reports the ground truth location of the actors without taking into account the simulated LiDAR in the simulation.

4 Detailed Implementation of Software Components

All code and analysis the author has written for the purposes of this project can be found at the following GitHub link: `https://github.com/SamKhzym/cloud-auto-park`.

The simulation platform used to simulate the dynamics of the physical twin and deploy all its relevant ROS2 nodes is the F1TENTH gym environment, which contains a simulated vehicle with a custom vehicle dynamics model (slightly higher fidelity than the one I implement for this project). It also contains a simulated 2D LiDAR, and odometry sensors that would be present on the real physical twin. It interfaces with other ROS2 components by publishing data to the `/ego_racecar/odom` topic and listening to the `/drive` for speed and steering angle messages. It also publishes its LiDAR scans to the `/scan` topic. It allows for the importing of custom maps which are implemented as single-channel grayscale images, where pixels higher than a specified threshold are treated as barriers that are detected by the simulated LiDAR.

The script `gen_parking_lot_map.py` was created as a configurable helper to generate parking lots of different sizes, with vehicle parked in different orientations and positions within the

parking space, allowing for more robust testing suites to be performed when this work is more mature. An example of a common testing environment used for the project with two vehicles parked near the center of two parking spaces is shown in Figure 2.

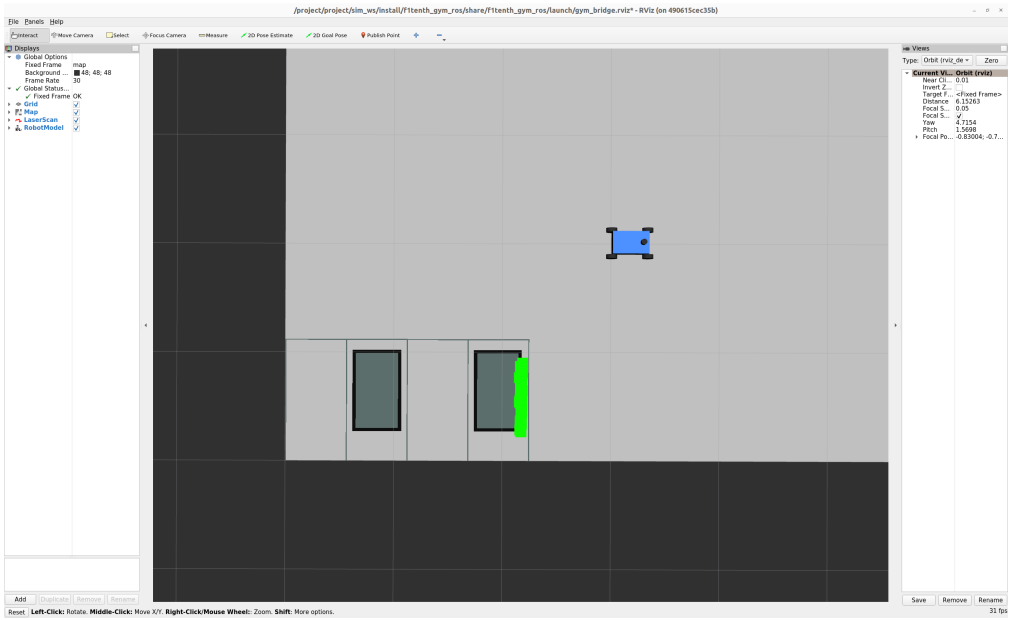


Figure 2: F1TENTH Gym Simulation Environment with Custom Parking Lot Map

4.1 Data Collection (Physical Twin)

The Data Collection component implementation is quite trivial and involves a ROS2 node listening to the `/ego_racecar/odom`, `/drive`, and `/scan` topics, then sending that information to the Data Management node on the digital twin (although this communication has been dropped since all communication interfaces are virtual for the purposes of the final submission). Since it is uncommon for vehicle applications to be working with a world reference frame, as the simulation environment provides, helper functions were written to convert the global x-y coordinates to a pair of GPS coordinates which are more commonly used in vehicle applications for global localization. This was done using the Haversine equations and deriving simplified inverse Haversine equations to go from GPS to global coordinates and vice versa. Note that the origin is an arbitrary initial GPS coordinate pair $(\phi_{ref}, \lambda_{ref})$ used as a reference for all other calculations. For all following equations, R represents the radius of the Earth.

Core Haversine Equation:

$$d_{hav}(\phi_1, \lambda_1, \phi_2, \lambda_2) = 2R \arcsin\left(\sqrt{\sin^2\left(\frac{\phi_2 - \phi_1}{2}\right) + \cos(\phi_1)\cos(\phi_2)\sin^2\left(\frac{\lambda_2 - \lambda_1}{2}\right)}\right) \tag{1}$$

Signed Haversine Equations (Conversion from GPS to Cartesian Coordinates):

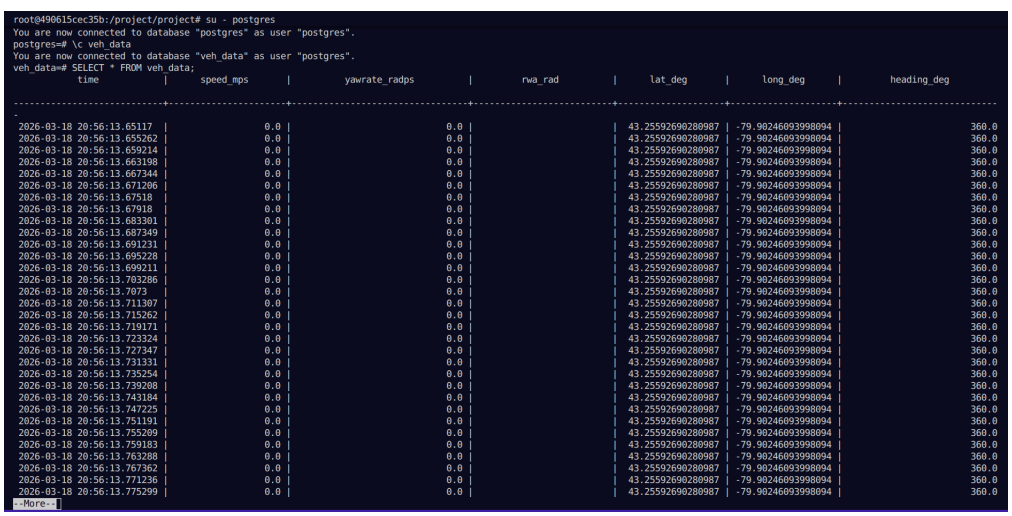
$$
\begin{aligned}
\Delta m_{lat} &= \operatorname{sgn}(\phi_{new} - \phi_{ref}) \cdot d_{hav}(\phi_{ref}, \lambda_{ref}, \phi_{new}, \lambda_{ref}) \\
\Delta m_{long} &= \operatorname{sgn}(\lambda_{new} - \lambda_{ref}) \cdot d_{hav}(\phi_{ref}, \lambda_{ref}, \phi_{ref}, \lambda_{new}) \\
x_{ego} &= \Delta m_{long} \sin(\theta) + \Delta m_{lat} \cos(\theta) \\
y_{ego} &= -(\Delta m_{long} \cos(\theta)) + \Delta m_{lat} \sin(\theta)
\end{aligned} \tag{2}
$$

Inverse Haversine Equations (conversion from Cartesian to GPS Coordinates):

$$
\begin{aligned}
\phi &= \frac{x_{ego}}{R} + \phi_{ref} \\
\lambda &= 2 \arcsin\left(\frac{\sin\left(\frac{-y_{ego}}{2R}\right)}{\cos(\phi_{ref})} \right) + \lambda_{ref}
\end{aligned} \tag{3}
$$

4.2 Data Management (Digital Twin)

The Data Management component recevies all sensor data and constantly pushes them to a PostgreSQL database called `veh_data` for later lookup. This database has fields for time, speed, yaw rate, road wheel angles, GPS latitudes, longitudes, and headings. This is shown in Figure 3. Because of the TimescaleDB extension, pushing and retrieving timeseries data is very efficient, allowing for the handling of odometer data coming in at roughly 200 Hz. This is also accomplished by using a buffering system and only pushing to the database once enough entries have been collected. This leaves room for potential data loss if terminated early, but allows for much more efficient batch writes. Helper functions were also written to filter the vehicle data by date and interpolate where no data was received, allowing for more complex data analysis to be done. These database utils are implemented in `database_manager.py`.



```
root@490615cec35b:/project/project# su - postgres
You are now connected to database "postgres" as user "postgres".
postgres=# \c veh_data
You are now connected to database "veh_data" as user "postgres".
veh_data=# SELECT * FROM veh_data;
            time            |      speed_mps      |          yawrate_radps          |         rwa_rad         |      lat_deg       |      long_deg      |         heading_deg
-----------------------------+---------------------+---------------------------------+-------------------------+--------------------+--------------------+-----------------------------
 2026-03-18 20:56:13.65117  |                 0.0 |                             0.0 |                         |  43.25592690280987 | -79.90246093998094 |                       360.0
 2026-03-18 20:56:13.655262 |                 0.0 |                             0.0 |                         |  43.25592690280987 | -79.90246093998094 |                       360.0
 2026-03-18 20:56:13.659214 |                 0.0 |                             0.0 |                         |  43.25592690280987 | -79.90246093998094 |                       360.0
 2026-03-18 20:56:13.663198 |                 0.0 |                             0.0 |                         |  43.25592690280987 | -79.90246093998094 |                       360.0
 2026-03-18 20:56:13.667344 |                 0.0 |                             0.0 |                         |  43.25592690280987 | -79.90246093998094 |                       360.0
 2026-03-18 20:56:13.671206 |                 0.0 |                             0.0 |                         |  43.25592690280987 | -79.90246093998094 |                       360.0
 2026-03-18 20:56:13.67518  |                 0.0 |                             0.0 |                         |  43.25592690280987 | -79.90246093998094 |                       360.0
 2026-03-18 20:56:13.67918  |                 0.0 |                             0.0 |                         |  43.25592690280987 | -79.90246093998094 |                       360.0
 2026-03-18 20:56:13.683301 |                 0.0 |                             0.0 |                         |  43.25592690280987 | -79.90246093998094 |                       360.0
 2026-03-18 20:56:13.687349 |                 0.0 |                             0.0 |                         |  43.25592690280987 | -79.90246093998094 |                       360.0
 2026-03-18 20:56:13.691231 |                 0.0 |                             0.0 |                         |  43.25592690280987 | -79.90246093998094 |                       360.0
 2026-03-18 20:56:13.695228 |                 0.0 |                             0.0 |                         |  43.25592690280987 | -79.90246093998094 |                       360.0
 2026-03-18 20:56:13.699211 |                 0.0 |                             0.0 |                         |  43.25592690280987 | -79.90246093998094 |                       360.0
 2026-03-18 20:56:13.703286 |                 0.0 |                             0.0 |                         |  43.25592690280987 | -79.90246093998094 |                       360.0
 2026-03-18 20:56:13.7073   |                 0.0 |                             0.0 |                         |  43.25592690280987 | -79.90246093998094 |                       360.0
 2026-03-18 20:56:13.711307 |                 0.0 |                             0.0 |                         |  43.25592690280987 | -79.90246093998094 |                       360.0
 2026-03-18 20:56:13.715262 |                 0.0 |                             0.0 |                         |  43.25592690280987 | -79.90246093998094 |                       360.0
 2026-03-18 20:56:13.719171 |                 0.0 |                             0.0 |                         |  43.25592690280987 | -79.90246093998094 |                       360.0
 2026-03-18 20:56:13.723324 |                 0.0 |                             0.0 |                         |  43.25592690280987 | -79.90246093998094 |                       360.0
 2026-03-18 20:56:13.727347 |                 0.0 |                             0.0 |                         |  43.25592690280987 | -79.90246093998094 |                       360.0
 2026-03-18 20:56:13.731331 |                 0.0 |                             0.0 |                         |  43.25592690280987 | -79.90246093998094 |                       360.0
 2026-03-18 20:56:13.735254 |                 0.0 |                             0.0 |                         |  43.25592690280987 | -79.90246093998094 |                       360.0
 2026-03-18 20:56:13.739208 |                 0.0 |                             0.0 |                         |  43.25592690280987 | -79.90246093998094 |                       360.0
 2026-03-18 20:56:13.743184 |                 0.0 |                             0.0 |                         |  43.25592690280987 | -79.90246093998094 |                       360.0
 2026-03-18 20:56:13.747225 |                 0.0 |                             0.0 |                         |  43.25592690280987 | -79.90246093998094 |                       360.0
 2026-03-18 20:56:13.751191 |                 0.0 |                             0.0 |                         |  43.25592690280987 | -79.90246093998094 |                       360.0
 2026-03-18 20:56:13.755209 |                 0.0 |                             0.0 |                         |  43.25592690280987 | -79.90246093998094 |                       360.0
 2026-03-18 20:56:13.759183 |                 0.0 |                             0.0 |                         |  43.25592690280987 | -79.90246093998094 |                       360.0
 2026-03-18 20:56:13.763288 |                 0.0 |                             0.0 |                         |  43.25592690280987 | -79.90246093998094 |                       360.0
 2026-03-18 20:56:13.767362 |                 0.0 |                             0.0 |                         |  43.25592690280987 | -79.90246093998094 |                       360.0
 2026-03-18 20:56:13.771236 |                 0.0 |                             0.0 |                         |  43.25592690280987 | -79.90246093998094 |                       360.0
 2026-03-18 20:56:13.775299 |                 0.0 |                             0.0 |                         |  43.25592690280987 | -79.90246093998094 |                       360.0
--More--
```

Figure 3: PostgreSQL Database for Incoming Vehicle Data

4.3 Manual Vehicle Controller (Physical Twin)

There are two distinct ROS2 nodes which allow for the manual control of the simulated physical twin. The first is the provided `teleop_twist_keyboard` ROS2 component which allows the user to press keyboard buttons to control the speed and steering angle of the physical twin. This provides helpful for exploratory, unstructured testing. The second implementation is a custom `manual_controller` node which causes the vehicle to drive in a swept sine pattern (both in the speed and steering domains). This allows for a rich amount of more structured data which allows exposure to many different combinations of speed and steering and can be used in data processing and optimization sweeps. An example of the vehicle's path with this script is shown in Figure 4.

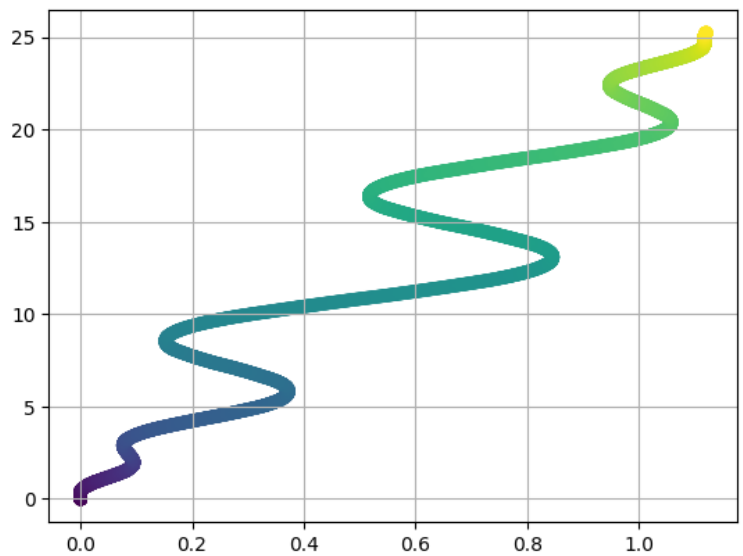


Figure 4: Path of Manually Controlled Physical Twin

4.4 Vehicle Parameter Optimizer (Digital Twin)

The Vehicle Parameter Optimizer component currently is not fully set up as a ROS2 node, but the core implementation exists in a Jupyter notebook called `process_driving_data.ipynb`. The core function of this component, and one of the major premises of including a cloud-based digital twin to this particular application, is that it allows the controller to be adaptable to changes in the system (mass or tire changes, etc.) long after the original software calibration set had been deployed.

The mathematical model used to simulate vehicle plant model dynamics for many model-based controllers is called the dynamic vehicle bicycle model, which treats each axle of the vehicle as an individual wheel. For accurate system modeling, this model relies on six vehicle parameters. These are the distances from the front and rear axles to the center of gravity (l_f, l_r), the front and rear tire cornering stiffnesses (C_f, C_r), the mass (m), and the moment of inertia about the vertical axis (I_z). The state update equation is shown in Equation 4, with the lateral velocity (v_y) and yaw rate (ω) being the state variables, and the vehicle's

road wheel angle (δ) as the input variable. The vehicle speed is assumed to be known from a different source and incorporated in the A matrix as v_x. This is integrated forward by using the Forward Euler method, shown in equation 5.

$$\underbrace{\begin{bmatrix} \dot{v_y} \\ \dot{\omega} \end{bmatrix}}_{\dot{\mathbf{x}}} = \underbrace{\begin{bmatrix} -\frac{C_f+C_r}{mv_x} & \frac{l_rC_r-l_fC_f}{mv_x}-v_x \\ \frac{l_rC_r-l_fC_f}{I_zv_x} & -\frac{l_f^2C_f+l_r^2C_r}{I_zv_x} \end{bmatrix}}_{A} \underbrace{\begin{bmatrix} v_y \\ \omega \end{bmatrix}}_{\mathbf{x}} + \underbrace{\begin{bmatrix} \frac{C_f}{m} \\ \frac{l_fC_f}{I_z} \end{bmatrix}}_{B} \delta \tag{4}$$

$$\mathbf{x}_{k+1} = \dot{\mathbf{x}}_k \cdot \Delta t + \mathbf{x}_k \tag{5}$$

The global vehicle pose can also be tracked using the state variables from the bicycle model and integrated using the Forward Euler method, shown in Equation 6.

$$\begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\psi} \end{bmatrix} = \begin{bmatrix} \cos\psi & -\sin\psi & 0 \\ \sin\psi & \cos\psi & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} v_x \\ v_y \\ \omega \end{bmatrix} \tag{6}$$

While the axle locations are unlikely to ever change, the mass distribution and tire dynamics could change if the driver decides to replace the tires, or if they are carrying something heavy that throws off the original distribution estimate. While these parameters may not deviate much from the original parameter set immediately, it is likely that the lifetime of a normal vehicle will cause enough parameter drift due to maintenance and repair will cause the accuracy of these parameters to drift, thus degrading model-based controller performance.

An online estimator for these parameters was created by taking a manual driving log over a sufficiently long period (1-2 minutes), and optimizing the C_f, C_r, m, I_z parameters to achieve the closest performance between predicted and measured variables over the yaw rate and vehicle path of the driving data. Two distinct root mean square error (RMSE) terms are used as metrics. The λ_ω and λ_{path} hyperparameters control which term is more influential in the cost function.

$$RMSE_\omega = \sqrt{\frac{1}{N}\sum_{i=1}^{N}(\omega_i - \hat{\omega}_i)^2} \tag{7}$$

$$RMSE_{path} = \sqrt{\frac{1}{N}\sum_{i=1}^{N}(x_i - \hat{x}_i)^2 + (y_i - \hat{y}_i)^2} \tag{8}$$

$$C_f^\star, C_r^\star, m^\star, I_z^\star = \underset{C_f, C_r, m, I_z}{\operatorname{argmin}} \left(\lambda_\omega \cdot RMSE_\omega + \lambda_{path} \cdot RMSE_{path}\right) \tag{9}$$

This optimization was only performed once for the purposes of this project, but a future version of this project should have it running periodically on more recent driving data to consistently update the parameters.

4.5 SLAM + Obstacle Detection (Digital Twin)

As mentioned in the addendum, this component has been de-scoped and does not perform any SLAM or obstacle detection yet. In the current iteration of the project, it just reports the ground truth centroids and orientations of the vehicles in the environment to the `/actors` topic to allow the planning and controls modules to perform the auto-park maneuver.

In future work, this component would receive data from the physical twin and perform SLAM and obstacle detection in the cloud to allow for less onboard compute and memory required on the physical twin.

4.6 Auto-Park Trajectory Planner (Digital Twin)

The Auto-Park Trajectory Planner component receives the obstacle map from the obstacle detection component. In its current iteration, it calculates the desired pose of the parking vehicle to be in-between two existing parked vehicles, although there should be additional work to this component to determine the best desired pose given the free space of the environment. It then uses the vehicle's immediate state as the reference pose, and constructs a cubic spline with control points placed in geometrically pre-determined locations to try and minimize sharp turns during the parking maneuver. An example of the control point selection and path generation is shown in Figure 5.

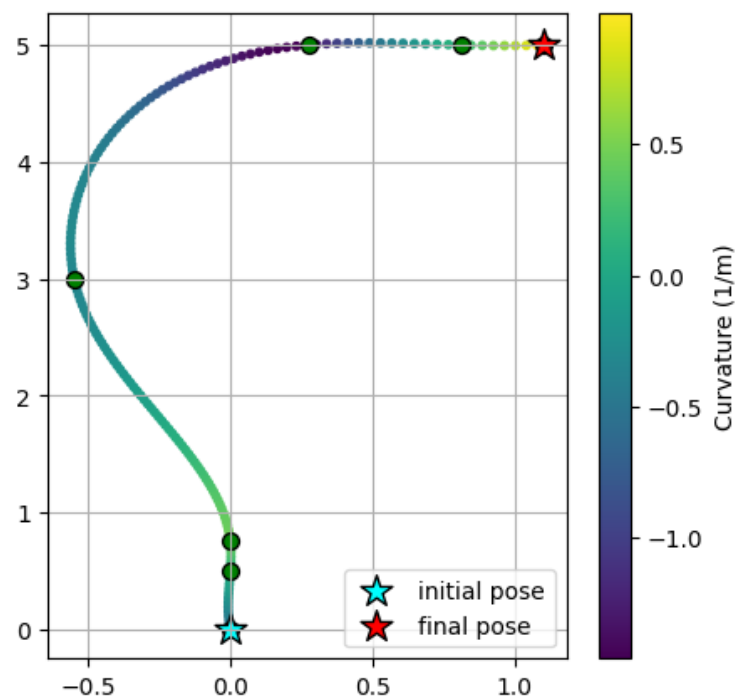


Figure 5: Generated Cubic Spline Trajectory with Control Points

The trajectory generation is done assuming a constant speed, and with the final trajectory quantized such that each control point corresponds to the desired vehicle pose at each controller step. It is also currently optimized for perpendicular parking and does not work well

on angled or parallel parking, which will be the exploration of future work as well. The implementation uses the `splprep` and `splev` implementations from `scipy`, which is convenient as it also allows to return the curvature of the path at every point.

This component is structured as a ROS2 service (`cloud_auto_park_srv.py`), which the client (physical twin) can call by providing the desired final vehicle pose (to be removed in a future version and replaced with outputs of the obstacle detector), the desired vehicle speed, and the controller sample rate. Once the trajectory generation is complete, it is packed as a `Trajectory` custom message as a list of points with an x-position, y-position, and curvature, and sent along with the optimal vehicle bicycle parameters back to the physical twin, implemented as the `BicycleModelParams` message.

4.7 Auto-Park Vehicle Controller (Physical Twin)

The Auto-Park Vehicle Controller receives the optimal vehicle bicycle model parameters and the desired trajectory from the digital twin, and uses a model-predictive controller (MPC) to determine the control actions for the physical twin. This involves solving an online optimization problem over a control horizon N_p to determine the optimal series of control actions δ_1 to δ_{N_p} with respect to a cost function $J(\delta_1, \ldots, \delta_{N_p})$. Regularization was also added to prevent the optimizer from saturating to the input bounds in more complex environments. The first control input δ_1 is then used as the command. The cost function used for this controller is purely position-based, taking the RMSE of vehicle path as the optimization objective, as shown in Equation 10.

$$\begin{aligned} \delta_1^\star, \ldots, \delta_{N_p}^\star &= \underset{\delta_1, \ldots, \delta_{N_p}}{\operatorname{argmin}} (RMSE_{path} + \lambda \cdot ||\delta||_2) \\ &\text{s.t.} |\delta_i| \leq \delta_{\max} \end{aligned} \tag{10}$$

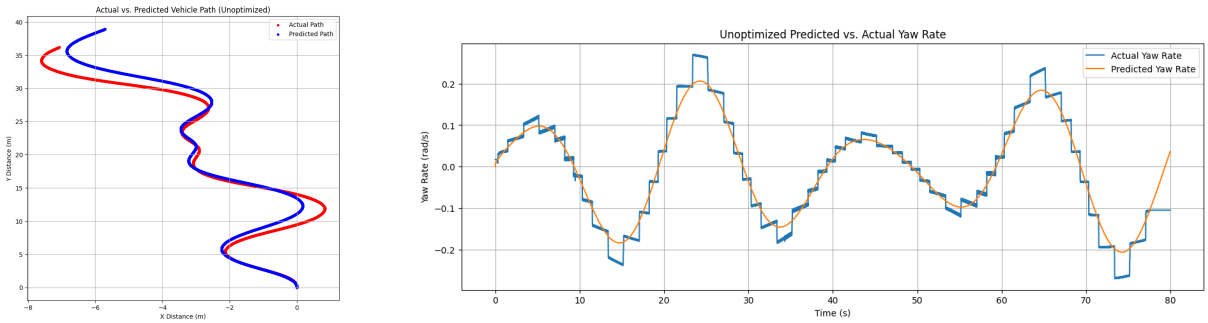
The `scipy.optimize.minimize` function was used with the SLSQP option for least squares optimization, allowing for relatively fast convergence for an online optimizer. The control horizon length N_p is a hyperparameter which was tuned to allow for quick convergence time while maintaining sufficient performance. The implementation of this component is in `mpc.py` and the ROS2 client which calls the trajectory planner service is implemented in `parking_controller.py`.

5 Results

5.1 Online Parameter Optimizer

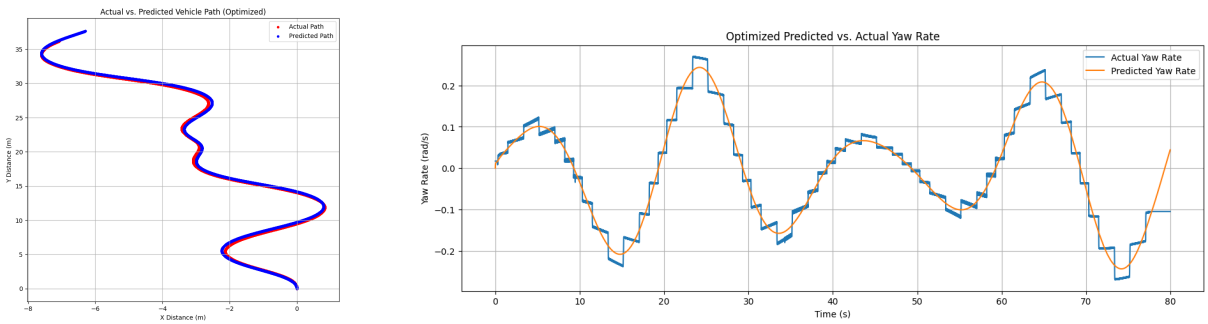
To start, an initial guess for vehicle bicycle model parameters was applied to a manual driving log. A note is that this simulation environment seems to provide weird artifacting in the yaw rate data that becomes exaggerated with higher speed. This does not seem to affect the results much, but is something to keep in mind for model inaccuracies.

The initial (unoptimized) vehicle parameter results (vehicle path and yaw rate) are shown



(a) Vehicle Path Comparison (b) Yaw Rate Comparison

Figure 6: Unoptimized Vehicle Parameter State Integration Comparisons



(a) Vehicle Path Comparison (b) Yaw Rate Comparison

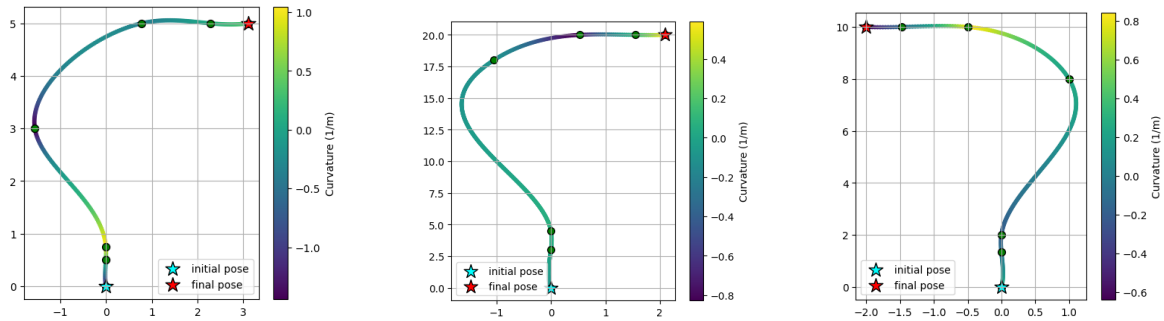
Figure 7: Optimized Vehicle Parameter State Integration Comparisons

in Figure 6, with an $RMSE_{path}$ of 0.639 m and an $RMSE_{\omega}$ of 0.0274 rad/s. The predicted path noticeably deviates substantially from the actual path, and the peaks of the yaw rate curve are larger in the actual vehicle data. The parameter optimizer ran for about 30 seconds over an 80 second driving log, and the final results are visibly much better as shown in Figure 7, with an $RMSE_{path}$ of 0.116 m and an $RMSE_{\omega}$ of 0.0228 rad/s. This technique seems to be effective to find optimal vehicle parameters and relatively computationally efficient considering how infrequently it would be run.

5.2 Trajectory Generator

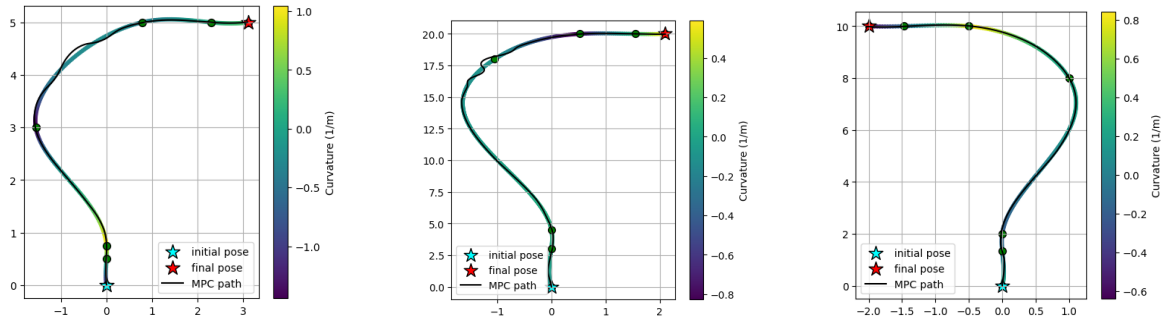
The trajectory generation technique used is rather simple, placing control points at geometrically "well-behaved" locations. These locations are heuristically picked, but they generally produce paths that have relatively low maximum absolute curvatures. This is desirable to avoid large changes in vehicle dynamics which generally perform worse with the MPC, and also avoids the possibility that the generated path is infeasible given the minimum turning radius of the vehicle.

Figure 8 shows three examples of perpendicular parking trajectories, with the green points representing the control points of the cubic spline, and the color corresponding to the curvature. In general, these results are good, with maximum curvature rarely going above 1 m^{-1}. However, these paths do not generalize to other types of parking, and the rules used to



(a) Desired Pose = (5, -3.1) (b) Desired Pose = (20, -2.1) (c) Desired Pose = (10, 2)

Figure 8: Trajectory Generation Examples for Different Desired Poses



(a) Desired Pose = (5, -3.1) (b) Desired Pose = (20, -2.1) (c) Desired Pose = (10, 2)

Figure 9: MPC Controller Evaluation on Different Desired Poses

place the control points cause highly suboptimal results if the desired pose is too close to the initial pose. To obtain better results in future work that are more generalizable for different kinds of paths, an optimization sweep could be performed at this stage of the problem to try to minimize the path's maximum absolute curvature, thus corresponding to potentially better performance of the final performance. This also could help with if there are additional obstacles in the original path, allowing the control points to move around any obstacles.

5.3 MPC Performance

The MPC was tested on several desired poses with a horizon of 10 timesteps. Overall, the performance was very good, almost never deviating significantly from the paths under test, as shown in Figure 9. However, there are a few issues with the current controller performance. The first are occasional oscillations in the traveled path, potentially due to few suboptimal optimization results for that segment. Furthermore, looking at the road wheel angle commands over time in Figure 10, there seem to be several small spikes in the output and larger peaks where the MPC saturates to the last value. This road wheel angle command high rate of change is not good in practical systems, so needs to be mitigated in future iterations of this work. This is a very promising base to start with, though.

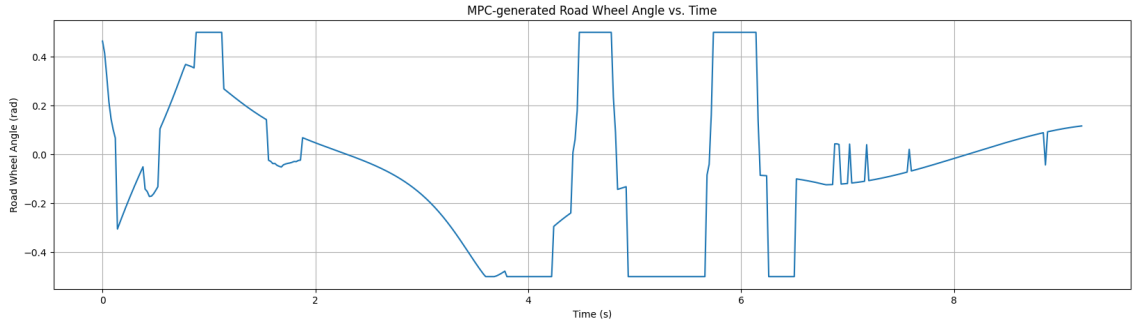


Figure 10: MPC Controller Evaluation on Different Desired Poses

6 Discussion and Future Work

Overall, I think this project was much larger than I initially anticipated, with many independent systems which have their own areas where they can be improved and optimized. However, given the complexity of the system, I am satisfied with the work that's been presented as an initial baseline and flexible framework to further improve and scale the systems presented in this project.

The vehicle parameter optimizer generally seems to be effective in improving vehicle dynamic modeling, but further consideration needs to be taken as to how often to run the optimizer and when to trust the results. For example, if the optimized results are still very poor, it could be indicative of an issue with the vehicle that is not accounted for in the bicycle model, and there should be a way for the system to avoid doing the parameter update and disallow the feature altogether. Furthermore, I would like to analyze how effective these optimization techniques are with slightly more sparse data (lower frequency and/or data drops).

The trajectory generator seems to perform well in a limited set of trial examples, and it is convenient to work with since it easily provides the curvature of the path, which is good to calculate "warm start" inputs for the MPC optimizer. However, it performs quite poorly on certain final poses and does not consider any obstacles in the path. I think an optimization sweep on the control points to minimize maximum absolute curvature and get as far away from obstacles as possible would be beneficial in the overall performance and potentially may scale to other parking types.

The MPC performs very well in the majority of cases, purely in terms of following the path. However, there is a lot of turbulence in the road wheel angle command, which may be due to occasional poor optimization convergence. An additional term in the cost function to penalize rate of change of the inputs may help with dealing with these oscillations. Adding RMSE of yaw rate over the horizon to the cost function as well might help the optimizer converge more easily and avoid these poorly-performing regions.

Finally, of course, I would like to see if this whole system will work on the physical units on the real physical twins, and what affects, if any, latency would have on the results. I think the appealing component of this digital twin application in the automotive is that the latency is generally tolerable since the vehicle is not moving while the calculation is happening in the

server. Since many automotive control tasks often require very tight deadlines and reliability which cannot be provided in a wireless medium, this is a unique application which lends itself well to distributed computing while still allowing for a more solid argument on system safety. Even if the control tasks are hard to generalize to other online control tasks that are more safety-critical, I do believe this online parameter estimation technique would be effective for a large family of algorithms which require accurate vehicle parameters, and thus vehicle data monitoring over a long time horizon as the vehicle changes and ages over time.